Defense slide plan

31 main slides, 12 backup. Target 35 minutes spoken. Rehearse to 33 so questions can start early and you have slack if a demo misbehaves.

Figures: most of what you need already exists in `thesis/figures/` (72 files) and in the running TilingAtlas app. Reuse rather than redraw. Where a slide needs something new it is marked NEW.

Timing per act is the budget, not a suggestion. If an act runs long in rehearsal, cut a slide from that act rather than borrowing from the proof.

Act 0: the question (2 slides, 2 min)

1. Title. Title, name, supervisor, date. Nothing else.

2. The numbers. One good tiling image, large. Underneath: 11, 20, 61, 151, 332, 673, 1472, 2850, …

Say: for each small k the number of these things is known. None of these numbers is proven. That is the thesis in one line, and everything after this slide is either how I tried to change that or what it cost.

Do not define anything yet. The picture carries it.

Act 1: what the objects are (4 slides, 4 min)

Discipline here decides whether you finish on time. Show, do not define. Every one of these slides is an image with two lines of text.

3. Tilings by regular polygons. Three or four examples. Edge-to-edge, all edges the same length. Say what those two constraints mean by pointing at a picture that violates them.

4. Vertex configuration. One tiling with two or three vertices labelled 3.12.12, 3.4.6.4, 3.6.3.6. Say: read the polygons around a vertex in order. That is the whole notation.

5. k-uniform. THE definition. Side by side: a 1-uniform tiling and a 2-uniform one, with vertex orbits coloured. Vertices fall into exactly k classes under the tiling's *own* symmetries.

This is the only definition that must land. Spend 90 seconds. If someone is lost after this slide they are lost for the rest of the talk. Do not mention wallpaper groups, orbits as a group-theoretic concept, or fundamental domains. Colour the orbits and say "these vertices are the same vertex as far as the tiling is concerned."

6. Who counted what, and the gap. Krötenheerdt 1969 (k=2), Chavey 1984 (k=3), Galebach 2002–2020 (k=4..7), Čtrnáct, Griffin and Kopczyński 2022 (k=8..15). Then the gap, in one line each: the small cases rest on case analyses nobody ever independently re-derived, and from k=4 up they rest on computer searches, none of which comes with an argument that it missed nothing.

Act 2: the journey (7 slides, 11 min)

Open with your own line from chapter 9: in a computer-assisted classification the path taken is itself evidence. That sentence licenses this entire act; say it explicitly.

Every architecture slide ends with a named obstruction, never with "it was slow." The obstruction is the result.

7. Architecture zero: wallpaper fitting. The plan: generate a seed patch, fit the fundamental domains of the 17 wallpaper groups onto its construction points, recover the tiling as the orbit of the fitted domain.

Died before implementation matured. Several groups (cmm, p2 among others) have *parametric* fundamental domains whose shape depends on the tiling being sought, so no fixed-shape matching can succeed in general.

What survived: symmetry does not construct a tiling, but it constrains its period lattice, and it can be detected exactly after the fact.

8. Architecture one: expand-and-extract. Grow each seed in the plane, stamping rigid copies at frontier vertices with exact isometries, out to a radius budget. Then search the expanded patch for two independent translations mapping it onto itself, and extract a fundamental cell.

It reproduced the 11 uniform tilings. It was not a failure in every respect.

9. How it died, and the prune I could not have. Died of measurement: hard 2-uniform seeds saturating a 410-deep search stack with ~730-tile patches and climbing leaf counts. Locally legal, non-periodic boundary variants proliferating without bound. Three optimisation campaigns invalidated overnight, because the problem was the architecture and not the constant factor.

The tempting repair: emit as soon as a patch certifies a period, and prune that branch. I proved it unsound instead of shipping it. A periodically completable patch can still extend non-periodically into a *different* valid tiling, so the prune silently drops tilings. Use `fig-unsound-prune` here; it is already drawn.

The crystallising line: growth has no sound stopping rule, so the period must be fixed before the search begins.

10. Architecture two: solve for the period. Enumerate the candidate period lattices before placing any tile, then fill each resulting torus exhaustively. What makes it finite is the bounded-weight theorem: the period lattice of any k-uniform tiling is generated by sums of a bounded number of unit edges. Every decisive comparison exact, in $Q(\zeta_{24})$, because a floating-point error here changes an integer and with it the count.

11. Learning not to trust a count. The first full $k=2$ run terminated cleanly and counted 23 against the reference 20. Diagnosing that took an entire phase. The pipeline had only ever found 19 distinct tilings: the 23 was an under-merged 19, an over-count and an under-count cancelling to a plausible-looking number.

The missing twentieth had a fundamental cell smaller than the rigid seed core, so the patch the fill was seeded with could never fit inside the very tiling it was meant to find.

Then 4.6.12: a tuned area bound of 16k silently dropped an Archimedean tiling at k=1, and the defect was masked for months by the duplicate-count bug of opposite sign. Two errors cancelling to the expected 11. It survived because the number it produced was the number I was expecting.

This is your best methodological slide. Do not rush it. The lesson is one sentence: a count can be wrong twice in opposite directions and still look right, so counts are validated per-tiling against an independent catalogue or not at all.

12. The wall at k=4, measured. Coverage was not the obstacle: the longest k=4 cell vector sits inside the proven pool reach and the largest cell area inside the proven bound. The wall is purely combinatorial.

Seeds explode 30–60×: 13,000–27,000 fillable seeds against 449 at k=3. And the fill stage binds: on a representative sample, 25 of 25 fills timed out at both a 15-second and a 30-second cap, with no cell completed and no sampled fill ever reaching the gate.

Say plainly: an intractability measurement is a decisive answer, and it is the one the experiment returned.

13. What survived the method that failed. The bounded-weight theorem and its small-k sharpening with attainment certificates. The exact arithmetic substrate. The certified 61 at k=3, per-tiling. The prohibited-prune registry. And a characterised negative result, which is a result.

Act 3: the pivot (2 slides, 3 min)

This is the hinge of the talk. Slow down.

14. The engine in my own repository. For months there had been a second solver in the repo, used only as an oracle whose job was to disagree with my pipeline when my pipeline was wrong. It was Čtrnáct's, found on GitHub. It had reproduced the whole sequence to k=16 in the time my pipeline was spending on k=3.

15. The decision. I was treating the faster engine as a checking tool and the slower one as the contribution, and the reason for that arrangement had nothing to do with the mathematics. The slower one was mine.

But the claim was never "I built a solver." It was "the canonical counts have no completeness proof, and here is one," and nothing in that sentence requires the algorithm to be my own. And Čtrnáct's algorithm, read properly, has exactly the same missing piece as everyone else's: it is described, it is not proven. So the gap I had set out to close was sitting under the faster engine, unclosed, and covering a great deal more ground.

Land it: the proof comes first, and it attaches to whatever machinery can actually carry it. That rule applies to methods too, and it is not sentimental about authorship.

Act 4: how the engine works (3 slides, 4 min)

Only enough for the proof to make sense. Resist explaining more.

16. Search gluings, not placements. An abstract vertex is a cyclic sequence of half-edges with angles between them, and nothing in it records where the vertex lies. Use `fig-engine-halfedges`. One abstract vertex stands for a whole orbit. So a k-uniform tiling, with infinitely many vertices in the plane, is assembled from at most k abstract ones, and the search space is finite by construction with no bound needing to be proven beforehand.

17. The four local rules. Mismatch, mirror break, lost trail, false closure. One small diagram each. Do not prove anything here; you are setting up Obligation 2.

18. The pipeline. solve → prune → develop, using `fig-engine-pipeline`. Only `develop` touches geometry, in $\mathbb{Z}[\zeta_{12}]$, exactly, and by then the search is over. That is the whole reason the engine is fast, and it is also where the hardest proof obligation lives.

Act 5: the proof (6 slides, 10 min)

19. Why a proof, when the numbers already agree. Two reasons, both concrete.

The sequence is not itself known to be correct, so agreement establishes only that two programs concur, neither carrying an argument, and no third program can break the circularity.

And searches of this kind do lose tilings. An earlier Python implementation of this very algorithm returned 2849 at $k=8$ and 5959 at $k=9$ against the correct 2850 and 5960. Off by exactly one, twice, detected only because an independent implementation disagreed. A search that omits one tiling in three thousand produces output inspection cannot distinguish from a correct one.

20. The theorem. `dev` is a bijection from emitted gluings onto k-uniform tilings. Termination, soundness, completeness, non-redundancy, one line each.

Then the tension, which is the reason the proof has content: soundness is easiest to secure by rejecting aggressively, completeness by rejecting nothing at all. The substance is showing the four rules reject exactly the assemblies that could never have become tilings, and not one assembly more.

21. The six obligations. The table from chapter 7, including the "whose" column. Obligation 4 is theirs. The rest are new.

You must be able to talk to this slide for two minutes with your back to it. This is the slide the grade is decided on.

22. Deep dive: Obligation 5, and why it is the hardest. The authors discharge it in one sentence. Show that sentence on the slide, quoted. Then the two failure modes no local rule can see: the walk returning with a different accumulated rotation (not flat), and tiles laid down with no local overlap that still overlap after wrapping around.

Then the route, with diagrams: glue one regular n-gon per face walk, get a compact flat oriented surface (flat because every vertex link closes at exactly 2π, oriented because every gluing reverses boundary orientations), apply Killing-Hopf to conclude it is a torus and nothing else, then observe that the plane covers a flat torus and a covering map of the plane by the plane is a homeomorphism, which is what excludes the global overlap.

The plain sentence to end on: we glued abstract pieces together with no coordinates anywhere, and we know the result is a picture rather than a broken one because the only closed flat oriented surface is a torus, and the plane wraps onto a torus without ever folding back on itself.

This slide is where you demonstrate the skill your professor is grading. Rehearse it more than any other.

23. Obligation 6, in ninety seconds. Nothing in the original paper addresses this, and without it the word completeness is void of content. The other five say the engine produces tilings and loses none it was capable of building. This one says the tilings it was capable of building are all the tilings there are.

The route in one breath: k-uniform gives a cocompact wallpaper group, hence a translation lattice, hence a finite quotient, and that quotient is itself a legal gluing over the alphabet with exactly k vertices. Note in passing that periodicity is a theorem here, not an assumption.

24. What it rests on, and how it could be wrong. Two classical inputs named. Four machine certificates, recorded as checks and not arguments. Then §7.6's four risks, in your order: implementation-versus-algorithm first, mathematics last, with Obligation 3 named as the likeliest hiding place.

Put your own sentence on the slide: I believe it is correct and I would not yet call the catalogue proven on my own authority.

Say it before anyone makes you say it. This slide converts your weakest point into your most credible one.

Act 6: results (3 slides, 3 min)

25. The counts. The table through k=16. Then immediately the honest framing, because it is stronger than the table: this sets no record. Čtrnáct, Griffin and Kopczyński published through k=15 and a later implementation reaches k=18. The table contains no tiling that was not already known. What it is, is a check on my re-implementation, and a good one, because a search that loses tilings tends to lose them at some particular k rather than uniformly.

26. Theorem-certified 11 and 20. The Delaney-Dress chain, which shares no machinery with either enumeration method and consults no prior catalogue: a flag-orbit bound makes the symbol sweep finite, a realizability lemma proven here puts minimal flat symbols in bijection with congruence classes of k-uniform tilings, and a terminating realizer turns every surviving symbol into a certified tiling. As far as I know this is the first mechanical verification of Krötenheerdt's 1969 count.

Also say where it stops: symbol generation walls at k=3.

27. The alphabet as an input, and what it found. Make the alphabet an input instead of a hard-wired table and the same unmodified search reaches star polygons, composite tiles, scaled families, polyominoes. Priority note in one clause: Čtrnáct's later implementation already had this and I reached it independently, not first.

On the star family: every in-ring entry of Myers's hand catalogues matched, 37 at k=1 and all 34 in-ring at k=2, plus four tilings his 2-uniform list does not contain, which survived three independent adversarial reviews and are reported as candidate omissions.

Then the scope line: these are an exhibition of the mechanism, not proven enumerations, because Obligation 1 does not transfer.

Act 7: close (2 slides, 2 min)

28. TilingAtlas. Live demo, 60 seconds, one tiling rendered from exact coordinates and one parameter moved. Have a recorded screen capture as the fallback and do not apologise if you use it.

29. What is mine and what is not. Put it on a slide, blunt, two columns. It is already §1.5 of the thesis. Doing this yourself removes the sharpest question in the room, and candour on this point reads as confidence rather than concession.

30. What is open. Prove the generator complete for one new tile family and that family's catalogue becomes as certified as the regular one. Certified enumeration at k=4. Peer review of the proof.

31. Closing slide. The number line from slide 2 again, now with the theorem underneath it. No thank-you slide, no questions slide. End on the content and stop talking.

Backup slides (12, unnumbered, after the end)

Build these. They are what make Q&A comfortable, and each one turns an ambush into a prepared answer. Flipping to a slide reads as depth; improvising reads as the opposite.

1. **The octagon theorem.** Statement plus the propagation argument, plus the one-line cost.
2. **Obligation 1 in detail.** 21 angle-valid configurations → six excluded species → 4.8.8 orphan → 14 configurations → 44 vertex types via the site-symmetry split.
3. **Obligation 2 in detail.** The four rules with the necessity argument, and the note that the divisor condition is the only rule reasoning about the quotient rather than the plane.

4. **Obligation 3 and the k=8 exhibit.** The no-drop lemma, then 2849 versus 2850, the one-dart root cause, and the A5/A6 certificates that exclude that failure class. Your best story; make sure this slide exists.
5. **Obligation 4 and why WL should not work.** The 3-cycle versus 6-cycle counterexample, then the frame-rigidity reason it works here anyway.
6. **The lemma ledger.** The full table from A.11. Answers "how much is actually proven" instantly.
7. **The bounded-weight theorem.** Statement and what it buys.
8. **The small-k sharpening.** Proven per-branch radii at $k \leq 3$, measured maxima 5, 6, 7, with attainment certificates, and the proven-pool re-run reproducing the catalogues with no oracle.
9. **The k=4 wall, full numbers.** Seed counts, timeout table, stage profile.
10. **Exact arithmetic.** Why $\mathbb{Z}[\zeta_{12}]$, why floats are unsafe here, and why ζ_{24} is only needed for the octagon.
11. **The prohibited-prune registry.** Both proven-unsound prunes, and the orbit-floor prune that fired zero times and why that negative result is evidence.
12. **The completeness guard.** `EU_NCBUDGET`, the shout-on-bind discipline, and the scaled-palette case where the default budget silently cost 73 tilings out of 1064 with the fixpoint only at budget 12.

Preparation order

Do these in this sequence. The slides are the last thing, not the first.

1. Learn the six obligations from `SIX_OBLIGATIONS.md` until you can say each claim, its failure mode, and its spine without looking. This is the load-bearing work and it is worth more than any slide.
2. Build slides 19–24 first, the proof act. If time runs out later, everything else is easier to improvise than this.

3. Build backup slides 1–6. They are mostly extracted from chapter 7 and appendix A and cost little.
4. Build acts 0–4, which are largely existing figures with captions.
5. Build acts 6–7.
6. Rehearse the whole thing aloud, timed, twice. Once alone with a stopwatch, once for someone who does not know the field. The second rehearsal is the one that finds the slides where you lose people.